Projet Air France

Lahmadi Hiba Le Tollec Alex Ouladsine Yasmine Carbillet Mathis
Huang Antoine

18/03/2026

Contents

1 Contexte **1**

2 Résolution avec maintenances : modèle $\mathbf{X}_{ijtk}$ **2**
2.1 Nouvelles données du problème . . . 2
2.2 Notations . . . 3
2.3 Principe de la modélisation . . . 3
2.4 Variables de décision . . . 3
2.5 Hypothèses de modélisation . . . 4
2.6 Construction des arcs valides . . . 4
2.7 Contraintes . . . 5
2.7.1 Couverture des vols . . . 5
2.7.2 Conservation de flot . . . 5
2.7.3 Nœud fictif — départ et retour . . . 5
2.7.4 Capacité de maintenance par aéroport et par jour . . . 6
2.7.5 Fréquence de maintenance . . . 6
2.8 Fonction objectif . . . 6
2.9 Résolution . . . 6
2.9.1 Paramètres de l'instance de référence . . . 6
2.10 Exemples de résultats . . . 7
2.11 Analyse des performances . . . 8

3 Modèle d'attribution $\mathbf{x_{it}}$ avec maintenances **9**
3.1 Variables . . . 9
3.2 Contraintes . . . 9
3.3 Fonction objectif . . . 10

4 Pistes d'amélioration **11**
4.1 Discussions des choix/hypothèses . . . 11
4.2 Gestion des Perturbations . . . 12

5 Analyse des performances des modèles **12**

6 Conclusion **14**

1 Contexte

Objectif

L'objectif de ce projet est de concevoir un modèle d'optimisation sous **Gurobi** pour l'affectation d'une flotte d'avions à un programme de vols donné. Nous exploitons un jeu de données simulant divers scénarios (avec ou sans maintenances), faisant varier trois paramètres :

- **Densité (d)** : densité des vols par appareil.

- **Nombre d'avions (p)** : flotte totale disponible.
- **Horizon (h)** : durée de la période d'affectation (en jours).

Le modèle vise à déterminer l'attribution optimale des avions afin de **minimiser le coût total d'exploitation** (vols et maintances compris), ce coût dépendant des caractéristiques propres à chaque avion affecté.

Modèles utilisés

Deux catégories de modèles vont être étudiées.

- Les modèles de réseaux en arc x_{ijt} où l'avion t enchaîne le vol i puis le vol j
- Les modèles d'attribution de tâches x_{it} où l'avion t fait le vol i

2 Résolution avec maintenances : modèle $\mathbf{X}_{ijtk}$

On étend à présent la version classique du problème en intégrant des **contraintes de maintenance** : chaque avion doit être envoyé en maintenance à intervalle régulier, dans un aéroport habilité, et la capacité d'accueil de ces aéroports est limitée.

2.1 Nouvelles données du problème

Les nouvelles instances contiennent en plus les données suivantes à prendre en compte:

- les aéroports habilités à la maintenance ;
- le jour de chaque vol;
- une matrice de coûts de maintenance ;
- les paramètres globaux (horizon de maintenance, capacité journalière pour la maintenance).

2.2 Notations

Symbole	Signification
$\mathcal{F}$	Ensemble des vols réels, $\lvert\mathcal{F}\rvert = N_f$
$\mathcal{T}$	Ensemble des avions, $\lvert\mathcal{T}\rvert = N_t$
$\mathcal{D}$	Ensemble des jours de l'horizon
$\mathcal{A}$	Ensemble de tous les aéroports
$\mathcal{A}_m$	Sous-ensemble des aéroports de maintenance
H	Horizon de maintenance (en blocs jours)
C_m	Capacité de maintenance journalière par aéroport
FICTIF	Nœud fictif représentant le dépôt/parking ($= N_f + 1$)
d	Durée d'une maintenance
i, j	Indices de vols ($i, j \in \mathcal{F} \cup \{\text{FICTIF}\}$)
t	Indice d'avion ($t \in \mathcal{T}$)
k	Indicateur de maintenance ($k \in \{0, 1\}$)
from(i)	Aéroport de départ du vol i
to(i)	Aéroport d'arrivée du vol i
dep(i)	Heure de départ du vol i (en minutes depuis J0)
arr(i)	Heure d'arrivée du vol i (en minutes depuis J0)
day(i)	Jour du vol i
base(t)	Aéroport de base de l'avion t
Cost$[i][t]$	Coût d'affectation du vol i à l'avion t
CostMaint$[t][a]$	Coût de maintenance de l'avion t à l'aéroport a

2.3 Principe de la modélisation

Le problème est modélisé comme un **problème de flot sur un graphe orienté** (comme c'était le cas pour le premier modèle X_{ijt}). Chaque avion t est associé à un chemin dans ce graphe, partant du nœud fictif (base de départ) et y revenant (parking final), en passant par une séquence de vols réels.

Les **nœuds** du graphe sont l'ensemble des vols $\mathcal{F}$ auquel on ajoute un unique **nœud fictif** FICTIF représentant à la fois le dépôt initial et le parking final de chaque avion.

Les **arcs** (i, j, t, k) représentent la possibilité pour l'avion t de passer du vol i au vol j, avec ou sans maintenance (selon k).

2.4 Variables de décision

La variable binaire centrale du modèle est :

$$x_{ijtk} \in \{0, 1\}$$

$x_{ijtk} = 1$ signifie que l'avion t effectue le vol i puis le vol j, avec une maintenance intercalée après le vol i si $k = 1$.

Indice	Signification
i	Vol précédent (ou FICTIF pour le premier vol)
j	Vol suivant (ou FICTIF pour le dernier vol)
t	Avion réalisant le vol
$k = 0$	Enchaînement sans maintenance
$k = 1$	Possibilité d'effectuer une maintenance après le vol i

Remarque.

Une modélisation envisagée avait été d'introduire une variable binaire $y_{tda} \in \{0, 1\}$ indiquant que l'avion t effectue une maintenance le jour d dans l'aéroport $a \in \mathcal{A}_m$. Cette approche se heurte cependant à un problème majeur : il faut ajouter un grand nombre de contraintes de **liaison** entre les variables y_{tda} et les vols réellement effectués, afin de garantir que la maintenance déclarée corresponde bien à un vol arrivant dans l'aéroport a le jour d, et que l'avion t soit effectivement présent sur place. Ces contraintes de couplage alourdissent fortement le modèle et prennent beaucoup plus de temps à la résolution.

La variable x_{ijtk} est au contraire **pratique** : le flag $k = 1$ est attaché directement à un arc (i, j, t), ce qui signifie que la maintenance est associée par construction au vol i qui arrive dans l'aéroport concerné. Aucune contrainte de liaison n'est nécessaire. Cette modélisation améliore significativement les performances du solveur.

2.5 Hypothèses de modélisation

- **Maintenances instantanées.** La durée de la maintenance est fixée ; elle se déroule la nuit, après le dernier vol du jour.
- **Maintenance en fin de journée uniquement.** Une maintenance ne peut avoir lieu qu'après le *dernier* vol d'une journée. En conséquence, x_{ijt1} n'est créée que si $\text{day}(j) > \text{day}(i)$ (le vol j est un vol d'un jour suivant).
- **Continuité physique.** Un avion ne peut enchaîner deux vols que si l'aéroport d'arrivée du premier est l'aéroport de départ du second, et si le vol suivant part *après* l'arrivée du vol précédent.
- **Critère de non blocage de l'avion** On ajoute une contrainte pour empêcher qu'un avion reste une journée entière sans ne rien faire. Un avion peut enchaîner deux vols que si le deuxième vol a lieu le même jour ou le lendemain du premier vol.
- **Respect de la base.** Le premier vol d'un avion t doit partir de $\text{base}(t)$.
- **O**n considère également qu'on connaît l'état initial de chaque avion (le nombre de jours depuis sa dernière maintenance) : on le prend en compte au moment de la vérification de la fréquence de maintenances.

2.6 Construction des arcs valides

Pour limiter le nombre de variables et accélerer la résolution, seuls les arcs *physiquement réalisables* sont créés.

De plus les vols sont triés par ordre croissant d'heure de départ : pour tout vol i, on ne teste comme successeur que les vols j tels que $\text{dep}(j) \geq \text{arr}(i)$, ce qui réduit considérablement l'espace de recherche.

Pour chaque avion t et chaque vol $i \in \mathcal{F}$, trois types d'arcs peuvent être générés :

1. Arcs réels $i \to j$.

- x_{ijt0} est créée si et seulement si :

$$\text{from}(j) = \text{to}(i) \quad \text{et} \quad \text{dep}(j) \geq \text{arr}(i) \quad \text{et} \quad \text{day}(i) + 1 \geq \text{day}(j)$$

(enchaînement physiquement possible, sans maintenance : c'est l'équivalent du x_{ijt} simple du modèle à 3 indices, avec l'ajout de la conditon que j est le même jour ou le lendemain de i)

- x_{ijt1} est créée seulement si, **en plus** :

$$\text{to}(i) \in \mathcal{A}_m \quad \text{et} \quad \text{day}(j) > \text{day}(i) \quad \text{et} \quad \text{dep}(j) \geq \text{arr}(i) + d$$

  (l'aéroport d'arrivée est habilité *et* le vol i est le dernier de sa journée pour cet enchaînement et le temps entre les deux vols est supérieur au temps de maintenance)

L'ajout de la contrainte **un avion ne peut pas rester un jour sans rien faire** permet de réduire grandement le nombre de variables du modèle. En effet, en son absence, le modèle crée les variables $x_{i,j,*,*}$ pour beaucoup de vols j très éloignés temporellement du vol i, et pour lesquels l'enchainement i-j ne sera jamais réalisé en pratique.

2. Arcs de fin $i \to$ FICTIF (retour au parking).

- $x_{i,\text{FICTIF},t,0}$ est créée **pour tout vol** $i \in \mathcal{F}$
- $x_{i,\text{FICTIF},t,1}$ est créée uniquement si $\text{to}(i) \in \mathcal{A}_m$ (une maintenance peut précéder le parking si on termine dans un aéroport de maintenance)

3. Arcs de départ FICTIF $\to i$ (départ de la base).

- $x_{\text{FICTIF},i,t,0}$ est créée uniquement si $\text{from}(i) = \text{base}(t)$ (le vol i peut être le premier vol de l'avion t)

Le stockage sous forme de `tupledict` Gurobi (clé = tuple (i, j, t, k)) permet d'utiliser la syntaxe `X.sum('*', j, t, '*')` avec des indices génériques '*', ce qui est à la fois plus rapide et plus compréhensible que des boucles explicites avec gestion des indices manquants.

2.7 Contraintes

2.7.1 Couverture des vols

Chaque vol réel doit être effectué **exactement une fois**, par un seul avion :

$$\sum_{j,\,t,\,k} x_{ijtk} = 1 \qquad \forall\, i \in \mathcal{F}$$

Cette contrainte garantit aussi qu'on ne peut pas avoir simultanément $x_{ijt0} = 1$ et $x_{ijt1} = 1$ pour un même triplet (i, j, t).

2.7.2 Conservation de flot

Pour chaque avion t et chaque vol réel k, le nombre d'arcs entrants (vols précédents) doit égaler le nombre d'arcs sortants (vols suivants) :

$$\sum_{i,\,m} x_{iktm} = \sum_{j,\,m} x_{ktjm} \qquad \forall\, t \in \mathcal{T},\ \forall\, k \in \mathcal{F}$$

Cette contrainte assure la **continuité** du plan de vol de chaque avion : si un avion arrive sur un vol k, il doit obligatoirement repartir sur un vol (réel ou fictif).

2.7.3 Nœud fictif — départ et retour

Chaque avion quitte sa base exactement une fois et y revient exactement une fois via le nœud fictif :

$$\sum_{j,\,k} x_{\text{FICTIF},j,t,k} = 1 \qquad \forall\, t \in \mathcal{T}$$

$$\sum_{i,\,k} x_{i,\text{FICTIF},t,k} = 1 \qquad \forall\, t \in \mathcal{T}$$

2.7.4 Capacité de maintenance par aéroport et par jour

Le nombre de maintenances effectuées dans un aéroport $a \in \mathcal{A}_m$ lors du jour d ne peut excéder la capacité journalière C_m :

$$\sum_{\substack{i,j,t \\ \text{-¿ to}(i)=a,\ \text{day}(i)=d}} x_{ijt1} \ \leq \ C_m \qquad \forall\, a \in \mathcal{A}_m,\ \forall\, d \in \mathcal{D}$$

Implémentation. On pré-groupe les arcs par clé (aéroport de maintenance, jour) effectivement réalisables dans un dictionnaire `arcs_par_maint` avant d'écrire les contraintes. Cela évite de tester toutes les combinaisons $(a, d) \in \mathcal{A}_m \times \mathcal{D}$ et de créer des contraintes inutiles..

2.7.5 Fréquence de maintenance

Chaque avion doit effectuer **au moins une maintenance** dans chaque fenêtre de H jours consécutifs :

$$\sum_{\substack{i,j \\ ,d_{\text{start}} \leq \text{day}(i) \leq d_{\text{end}}}} x_{ijt1} \ \geq \ 1 \qquad \forall\, t \in \mathcal{T},\ \forall\, d_{\text{start}} \in \{0, \ldots, |D| - H\}$$

où $d_{\text{end}} = d_{\text{start}} + H - 1$.

Cette contrainte est générée en itérant sur chaque avion t. Afin d'intégrer **l'état initial** de l'appareil, la première fenêtre d'observation débute à une date antérieure au premier jour de planification :

- On définit un δ comme le nombre de jours écoulés depuis la dernière maintenance de l'avion t.
- La première fenêtre glissante est initialisée avec un décallage de δ.
- **Exemple** : Si un avion a effectué sa dernière maintenance 3 jours avant le début du planning et que l'horizon de maintenance est de 5 jours la première fenêtre de vérification couvrira l'intervalle sera $[-2, -1, 0, 1, 2]$, puis $[-1, 0, 1, 2, 3]$ ...

Cette approche permet de lier l'historique de l'avion aux décisions futures, assurant ainsi la continuité du cycle de maintenance.

2.8 Fonction objectif

On minimise la somme des coûts de vol et des coûts de maintenance :

$$\min \quad \underbrace{\sum_{\substack{i \in \mathcal{F} \\ j,\, t,\, k}} x_{ijtk} \cdot \text{Cost}[i][t]}_{\text{vol}} \ + \ \underbrace{\sum_{\substack{i \in \mathcal{F} \\ j,\, t,\, k=1}} x_{ijt1} \cdot \text{CostMaint}[t][\text{to}(i)]}_{\text{maintenance}}$$

Le terme $\text{Cost}[i][t]$ est indexé sur le vol i uniquement (pas sur j), car le coût est indépendant du vol suivant.
Le coût $\text{CostMaint}[t][\text{to}(i)]$ dépend de l'avion t et de l'aéroport $\text{to}(i)$ où est réalisée la maintenance.

2.9 Résolution

2.9.1 Paramètres de l'instance de référence

Le tableau ci-dessous résume les paramètres de l'instance utilisée pour le test suivant (`density=1, p=10, h=15, test_0`).

Paramètre	Valeur
Nombre de vols (N_f)	408
Nombre d'avions (N_t)	10
Nombre de jours ($\|D\|$)	15
Horizon de maintenance (H)	8 jours
Capacité de maintenance (C_m)	4 maintenances/jour/aéroport
Aéroports de maintenance	HAM, LGW, MAD, FRA, BCN, MUC, MXP, NCE, GLA, DUB, HEW

Pour cette instance on obtient:

Éléments	Quantité
Variables binaires x_{ijtk}	181 092
Contraintes	4 726

Table 1: Taille du modèle après construction.

La génération sélective des arcs réduit fortement le nombre de variables : seuls les enchaînements physiquement réalisables sont modélisés. Cela améliore les performances de Gurobi en réduisant le domaine de recherche dès la construction du modèle, avant même le presolve.

2.10 Exemples de résultats

Planning global de la flotte

La figure suivante présente le planning complet de la flotte. Chaque ligne représente un avion ; les segments bleus sont des vols ; les étoiles rouges indiquent une maintenance. Les barres colorées sous chaque avion matérialisent les intervalles entre maintenances (vert : intervalle $\leq H$; rouge : violation).

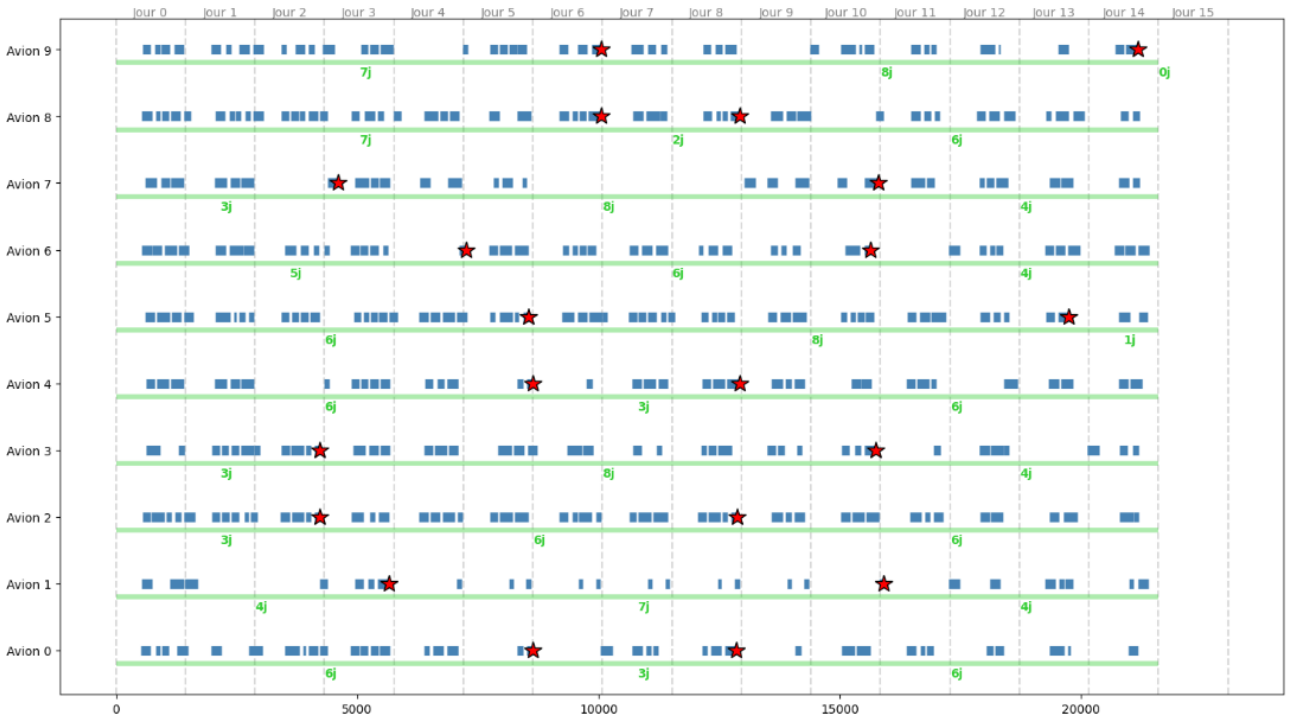
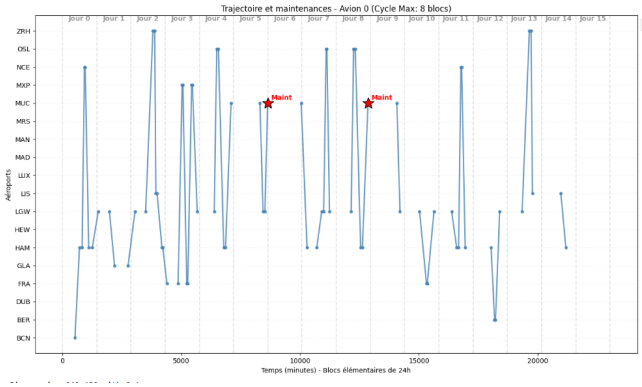


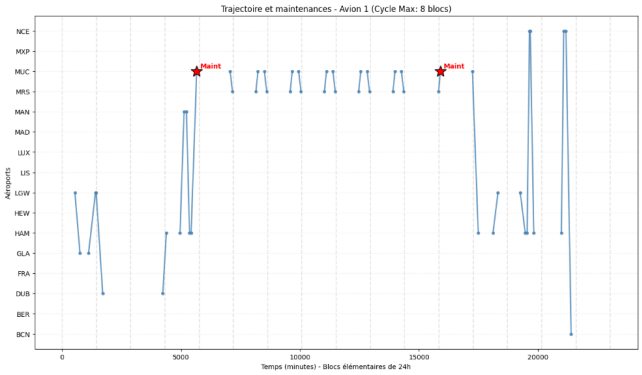
Figure 1: Planning global — toute la flotte sur l'horizon complet.

Plans de vol détaillés — exemples d'avions

Les figures suivantes montrent le plan de vol détaillé de deux avions. L'axe Y représente les aéroports ; l'axe X le temps en minutes. Un arc bleu est un vol ; une étoile rouge marque une maintenance.



(a) Trajectoire de l'avion 0.

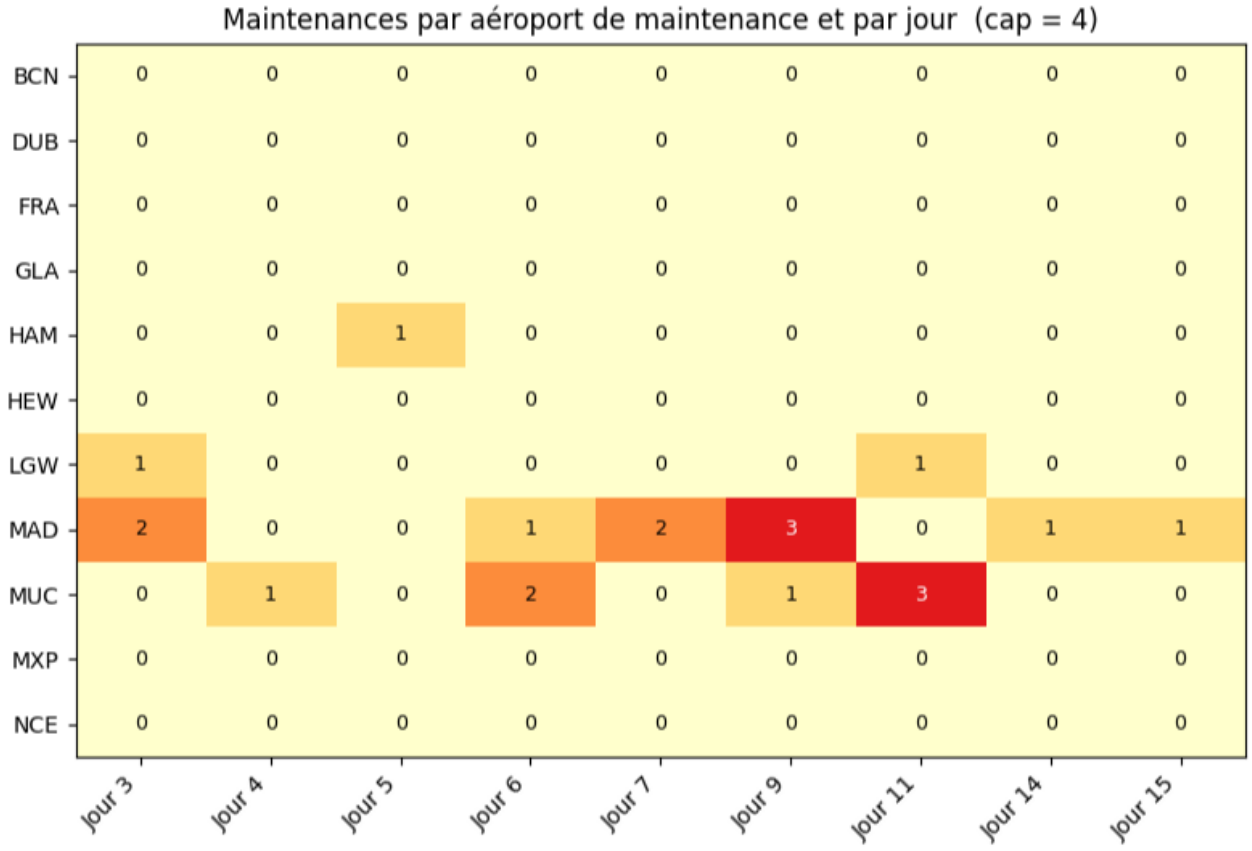


(b) Trajectoire de l'avion 1.

Figure 2: Plans de vol détaillés pour deux avions.

Maintenances par aéroport de maintenance et par jour

La figure suivante indique le nombre de maintenances effectuées à chaque aéroport et chaque jour d.



2.11 Analyse des performances

Ce modèle est plutôt efficace sur les petites instances. Quand on augmente le nombre total de jours les performances restent satisfaisantes. Cependant quand on augmente le nombre d'avions et de vols, les performances chutent rapidement a cause d'un nombre trop important de variables.

Avions (p)	Horizon (h)	Temps de résolution	Nombre de variables moyen
10	15	60 à 90 s	38000
10	20	100 s	43000
10	30	90 s	47000
20	15	200 à 500 s	210000
30	15	3000 à 13000 s	500000

Table 2: Temps de résolution selon les paramètres de l'instance (`density=1`, `d=4`).

3 Modèle d'attribution x_{it} avec maintenances

3.1 Variables

On considère comme variable de décision :

$$x_{it} = \begin{cases} 1 & \text{si l'avion } t \text{ réalise le vol } i \\ 0 & \text{sinon} \end{cases}$$

$$y_{dt} = \begin{cases} 1 & \text{si l'avion } t \text{ réalise une maintenance le jour } d \\ 0 & \text{sinon} \end{cases}$$

$$z_{it} = \begin{cases} 1 & \text{si l'avion } t \text{ réalise le vol } i \text{ qui le mène à une maintenance} \\ 0 & \text{sinon} \end{cases}$$

On a comme paramètres :

Paramètres
Nombre de vols (N_f)
Nombre d'avions (N_t)
Nombre de jours ($\|D\|$)
Nombre de jours fictifs (jours avant la simulation) ($\|D_{fictif}\|$)
Horizon de maintenance (H)
Capacité de maintenance (par jour/aéroport) (C_m)
Aéroports de maintenance

3.2 Contraintes

Couverture des vols: La contrainte

$$\sum_{t \in T} x_{it} = 1 \quad \forall i \in F$$

impose que chaque vol soit affecté à exactement un avion. Elle garantit que l'ensemble du programme de vols est couvert sans duplication ni omission.

Continuité des avions (contrainte de flot): Les contraintes de type

$$\sum_{\text{arrivées avant } i} x_{jt} - \sum_{\text{départs avant } i} x_{jt} \geq x_{it}$$

traduisent la conservation de flot pour chaque avion et chaque aéroport et imposent qu'un avion ne peut effectuer un vol i au départ d'un aéroport que s'il y est déjà présent, et assurent :

- la **continuité spatiale** (l'avion doit être au bon aéroport),
- la **continuité temporelle** (il doit arriver avant le départ du vol suivant).

Cas des aéroports initiaux: Pour les aéroports de base des avions, la contrainte devient :

$$\sum_{\text{arrivées}} x_{jt} - \sum_{\text{départs}} x_{jt} \geq x_{it} - 1$$

Le terme -1 permet d'initialiser la séquence : un avion peut effectuer son premier vol sans avoir effectué de vol précédent, à condition que ce vol parte de son aéroport initial.

Gestion des doublons: On définit l'ensemble des paires de vols redondants $\mathcal{D}$ par :

$$\mathcal{D} = \{(id_j, id_i) \in \text{Flights}^2 : i > j \wedge \text{attr}(f_i) = \text{attr}(f_j)\}$$

où le vecteur d'attributs est défini par :

$$\text{attr}(f) = (\text{departure}, \text{arrival}, \text{origin}, \text{destination})$$

Conditions initiales de maintenance: Soit $d_m(t)$ le jour de maintenance initialement assigné à l'avion t (jour avant le début de notre dataset). On fixe les variables de décision de maintenance y_{dt} selon :

$$y_{dt} = \begin{cases} 1 & \text{si } d = d_m(t) \\ 0 & \text{sinon} \end{cases} \quad \forall t \in \mathcal{T}, \forall d \in \mathcal{D}_{\text{fictif}}$$

Contrainte de fenêtre de maintenance: Afin de respecter les cycles de maintenance réglementaires, chaque avion $t \in \mathcal{T}$ doit effectuer au moins une opération de maintenance sur toute période glissante de durée H (horizon de maintenance) :

$$\sum_{d=d_0+1}^{d_0+H} y_{dt} \geq 1, \quad \forall t \in \mathcal{T}, \forall d_0 \in \mathcal{D} \mid d_0 + H \in \mathcal{D} \tag{1}$$

Séquence de maintenance de fin de journée: Pour garantir que la maintenance s'effectue après le dernier vol opérationnel, si un avion t est assigné à une maintenance suite au vol i ($z_{it} = 1$), il ne peut réaliser aucun vol ultérieur j le même jour d :

$$z_{it} + x_{jt} \leq 1, \quad \forall t \in \mathcal{T}, \forall d \in \mathcal{D}, \forall (i,j) \in \mathcal{F}_d^2 : \text{dep}_j > \text{arr}_i \tag{2}$$

Acheminement vers la maintenance Pour qu'une opération de maintenance soit validée le jour d pour l'avion t ($y_{dt} = 1$), l'appareil doit impérativement effectuer un vol j arrivant dans l'un des aéroports du réseau de maintenance $\mathcal{A}_m$:

$$\sum_{k \in \mathcal{A}_m} \sum_{j \in \mathcal{F}_{k,d}} z_{jt} = y_{dt}, \quad \forall t \in \mathcal{T}, \forall d \in \mathcal{D} \tag{3}$$

Cette égalité garantit qu'une maintenance ne peut être déclenchée sans vol d'apport, et réciproquement, qu'un vol vers la maintenance implique l'immobilisation de l'appareil pour révision.

Capacité des centres de maintenance Chaque aéroport du réseau de maintenance $k \in \mathcal{A}_m$ dispose d'une capacité d'accueil limitée par jour d. Le nombre total d'avions entrant en révision ne peut excéder la capacité C_k du centre :

$$\sum_{t \in \mathcal{T}} \sum_{i \in \mathcal{F}_{k,d}} z_{it} \leq C_k, \quad \forall k \in \mathcal{A}_m, \forall d \in \mathcal{D} \tag{4}$$

3.3 Fonction objectif

L'optimisation se fait en deux phases.

Phase 1 : Minimisation des Coûts Opérationnels

L'objectif primaire est de minimiser la somme des coûts d'assignation des vols et des coûts logistiques de maintenance. La fonction objectif Z_1 est formulée comme suit :

$$\min Z_1 = \sum_{i \in \mathcal{F}} \sum_{t \in \mathcal{T}} c_{it} x_{it} + \sum_{i \in \mathcal{F}_m} \sum_{t \in \mathcal{T}} \gamma_{t,k(i)} z_{it} \tag{5}$$

Où :

- c_{it} : Coût d'opération du vol i par l'appareil t.
- $k(i)$: Aéroport de destination du vol i (centre de maintenance potentiel).
- $\gamma_{t,k}$: Coût spécifique de maintenance pour l'avion t au centre de maintenance k.

Configuration du solveur : Pour garantir une convergence rapide vers une solution industriellement acceptable, le solveur **Gurobi** est paramétré avec un `MIPGap` de 2%.

Phase 2 : Optimisation de la Disponibilité Flotte

Une fois la valeur optimale Z_1^* obtenue, une stratégie de fixation de variables est appliquée. Pour chaque couple (i, t), si le vol i a été assigné à l'avion t en Phase 1, ses bornes sont restreintes :

$$x_{it}^{lb} = 1, \quad x_{it}^{ub} = 1 \quad \forall (i,t) \in \{(i,t) \mid x_{it}^* > 0.5\} \tag{6}$$

Une contrainte d'admissibilité est ajoutée pour garantir que le coût total ne dégrade pas l'optimum Z_1^*.

$$\sum_{i \in \mathcal{F}} \sum_{t \in \mathcal{T}} c_{it} x_{it} + \sum_{i \in \mathcal{F}_m} \sum_{t \in \mathcal{T}} \gamma_{t,k(i)} z_{it} \leq Z_1^* \tag{7}$$

Le second objectif Z_2 vise alors à optimiser le volume total de maintenance sur l'horizon :

$$\min\ Z_2 = \sum_{d \in \mathcal{D}} \sum_{t \in \mathcal{T}} y_{dt} \tag{8}$$

4 Pistes d'amélioration

4.1 Discussions des choix/hypothèses

Choix du GAP

On définit Sol_c la solution optimale du problème continu et Sol_i un des meilleurs éléments admissibles trouvé par l'algorithme. On a

$$\text{GAP} = \frac{|\text{Sol}_i - \text{Sol}_c|}{\text{Sol}_c}$$

Le choix du GAP est important car un GAP très proche de 0% augmente considérablement le temps de recherche du branch-and-bound (plateau atteint à 1% par exemple). On a estimé que les incertitudes des coûts d'entretien, d'amortissement, d'équipages, trajets/ carburants comptaient en tout pour environ 1%. Le but de ce projet est de tester significativement plusieurs modèles différents selon les différents data set disponible, on a pas trouvé pertinent de baisser ce GAP en dessous de 2%. En conditions réelles, avec un seul carnet de vol à attribuer, on pourra améliorer ce GAP si besoin.

Choix des paramètres

Gurobi propose plusieurs paramètres à adapter selon notre problème (MIPFocus, Presolve, Heuristics...)

Après plusieurs tests, on a constaté que donner une condition initiale gloutonne n'améliore pas/déteriore la recherche (la solution optimale n'est pas dans le voisinage d'une condition initiale qu'on pourrait calculer), les autres choix que MIPFocus = 0 ne sont pas pertinents. Le choix du coefficient pour les heuristiques n'a pas été jugé utile car on avait des résultats trop aléatoire dessus.

Le paramètre qu'on a choisi (fonctionne sur tout les data testés) est de mettre Presolve = 2 (réduction en plus de 10% minimum sur le nombre de contraintes). Cela peut s'expliquer par le fait que la mémoire de l'ordinateur peut-être un élément limitant l'efficacité du code.

Amélioration de la data

On a remarqué qu'il y avait des vols doublons $i \neq j$ (même aéroports de départs et d'arrivés, même temps de départ et d'arrivé). Cela n'arrive que dans des cas particuliers dans la vie réelle. Le problème c'est que cela introduit des variables $x[i,t]$ et $x[j,t]$ très semblables, donc ça donne des difficultés au solveur de trancher entre les deux. Une proposition serait de filtrer ces vols doublons.

4.2 Gestion des Perturbations

Fenêtre de Ré-optimisation

Lorsqu'un problème est détecté sur un vol, nous définissons une fenêtre de récupération $\Delta = [t_0, t_0 + \delta]$. Le modèle restreint alors l'espace de recherche aux variables impliquées dans cette fenêtre, tandis que le reste du programme de vol est maintenu par la contrainte de fixité :

$$x_{it} = x_{it}^{\text{init}} \quad \forall i \in \mathcal{F} \setminus \mathcal{F}_\Delta, \forall t \in \mathcal{T} \tag{9}$$

Traitement des Retards : Absorption et Swaps

Le modèle arbitre entre plusieurs leviers logiques pour stabiliser le réseau :

- **Absorption :** Utilisation des temps de rotation (buffers) pour éponger les retards mineurs sans affecter la ponctualité des vols suivants.
- **Aircraft Swaps :** Échange dynamique d'appareils entre deux rotations pour isoler une perturbation et éviter sa propagation sur des lignes critiques.
- **Propagation :** En dernier recours, le retard est propagé le long de la rotation de l'appareil, le solveur cherchant alors le point de convergence le plus proche pour un retour au planning nominal.

5 Analyse des performances des modèles

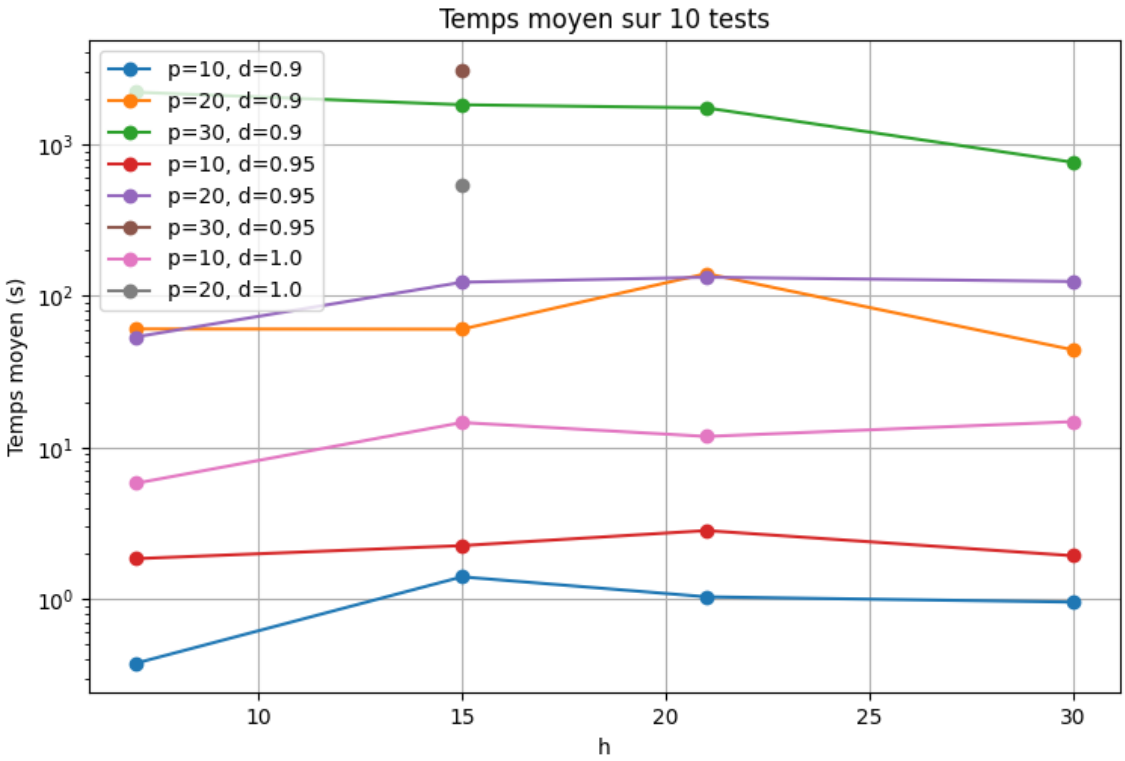
Le but de cette section est d'analyser les différents modèles pour faire un choix parmis ces modèles et comment les améliorer.

Dans toute cette partie, on utlise le benchmark suivant :

- ARM Apple M1 8 Core 3200 MHz
- Multi-Processing (1 Core par modèle)

Chaque processeur est utilisé envion à 80%.

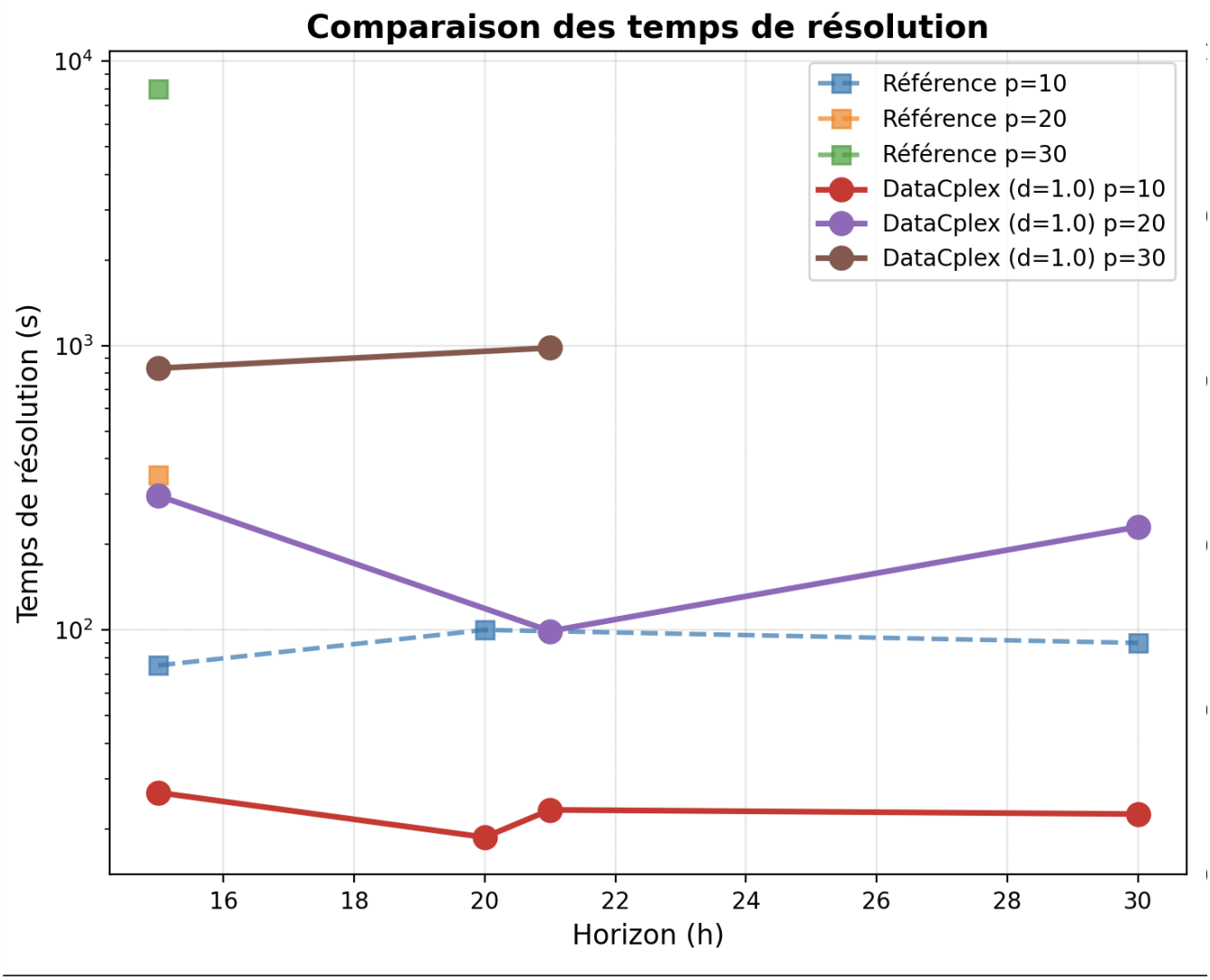
Modèle x_{it} sans maintenance



Le modèle ne converge pas pour le cas avec maintenances.

Grâce à une meilleure optmisation de gurobi (moins d'interdépendances entre Python et Gurobi) on a pu implémenter le modèle 2D (10 fois plus vite environ) et le modèle a pu terminer avec les maintenances cette fois ci :

On remarque que le modèle est très peu sensible à l'horizon (le nombre d'avion) mais le facteur critique est le nombre d'avion (facteur 10 à chaque fois qu'on augmente de 10 les avions)



Ici la réference, c'est le modèle x_{ijt}. On remarque qu'ils sont pas très différent en performance pour les petites instances mais qu'il y a une différence nette lorsqu'on fait sur 30 avions.

6 Conclusion

On a un modèle qui pourrait être un candidat intéréssant pour être testé sur des instances plus conséquentes et sur un serveur plus performant. (Le but est d'avoir un programme qui termine en 1 jour par exemple pour une prédiction de 1 semaine par exemple).